Project Vouch

Project one-liner

A restaurant review application geared towards the casual user who doesn't like spend 5-15 minutes on Google reviews

Motivation of development

Restaurant reviews are broken for the following reasons:

- Reviews target highly nerdy society which likes to work in lists and lengthy reviews
- Review systems at Google can be usually gamed / scammed
- Oftentimes the review system is not tailored to your taste principles. E.g. McDonalds can get >4 stars but you have other taste

Application principles

In order to make Recommendations easy and accessible for the casual everyday user, Project Vouch builds on the following principles:

Principle	**Explanation**
Giving a review should be as easy fast as possible	No stars, no lengthy texts and only focussing on the positive side will ensure that process stays lean
Giving a review should be as fast as possible	Data and location should be always pre-populated to save on time and reduce hurdles. The journey to recommendation should be a 1-3 click experience.
Your friends are the best gauge of your taste	Experience is closed. Instead of the whole world, recommendations are only limited to your network. This ensures that taste resembles recommendation and system is not gamed
Leverage AI and technology to work on first three principles	Technologies like location capturing, pre-populating locations, location notifications, action button should be leveraged in order to make the experience effortless

MVP Definition

MVP should focus on just basic needs of giving and finding recommendations plus very some functionality around the ease of process. The goal of the MVP is to get an application out in order to test the whole business model, flow and market.

Function	Explanation
Sign-In	Like Tinder this needs to be Phone Number Based
Log-In	Via Number / Connect Number to Apple, Google Account for fast login
Profile / User	Create a user, profile and make changes to profile
Location Tracking	User location tracking based on defined rules
Search	Search Google Maps for Restaurants close by
Location Approximator	Easy Algorithm that determines which restaurant you are most likely to be in right now (i.e. a simple weighted algorithm according to popularity, stars, size, location)
Popular near me	Easy Algorithm that determines which popular restaurants are near you right now (i.e. a simple weighted algorithm according to popularity, stars, size, location etc). Essentially these are the #2, #3 results from the Location Approximator
Recommendation	Actual process of setting a pin in the backend which marks the restaurant as recommended by user X. Recommendation has only two dependencies. User and List. Lists are defined lists which filter recommendations.
Google Maps API	Scrapping information from Maps like stars, photos, opening hours, price range etc.
Finding Recommendations	Dependent on the user contacts and set pins by his contacts, user can only find recommendations made by his contacts
Guide Mode Recommendations	Switching into guide mode will open pre-populated and curated recommendations by famous guides
Start Directions	Tbd -> handoff to Maps
Feedback	Users have text field to input feedback right to us
Rate Us	App store rating shortcut displayed in the app

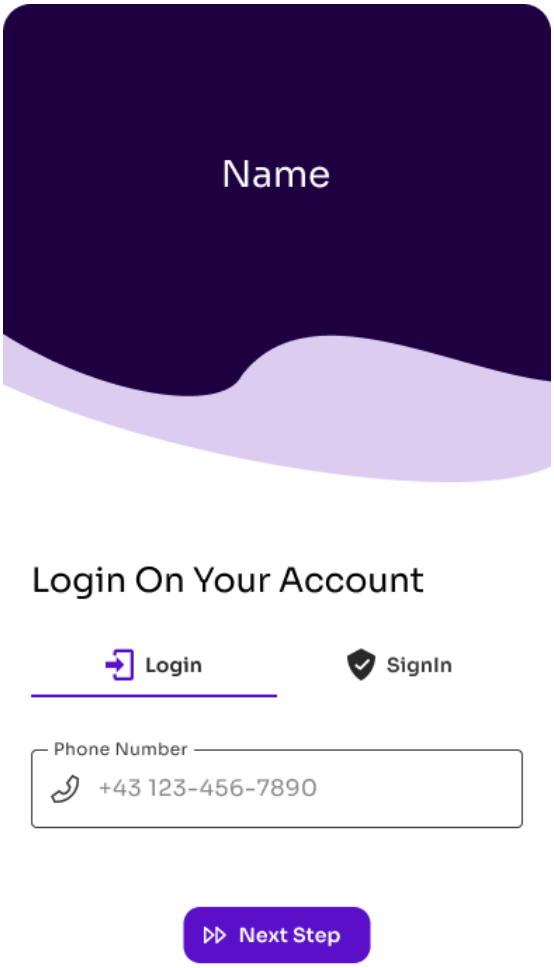
Language	Make app available in English, German, Spanish, French
Sent Push Notifications	Sent push notifications on specified IF THEN use cases

Front End Mock-Screens / Mock-Functions

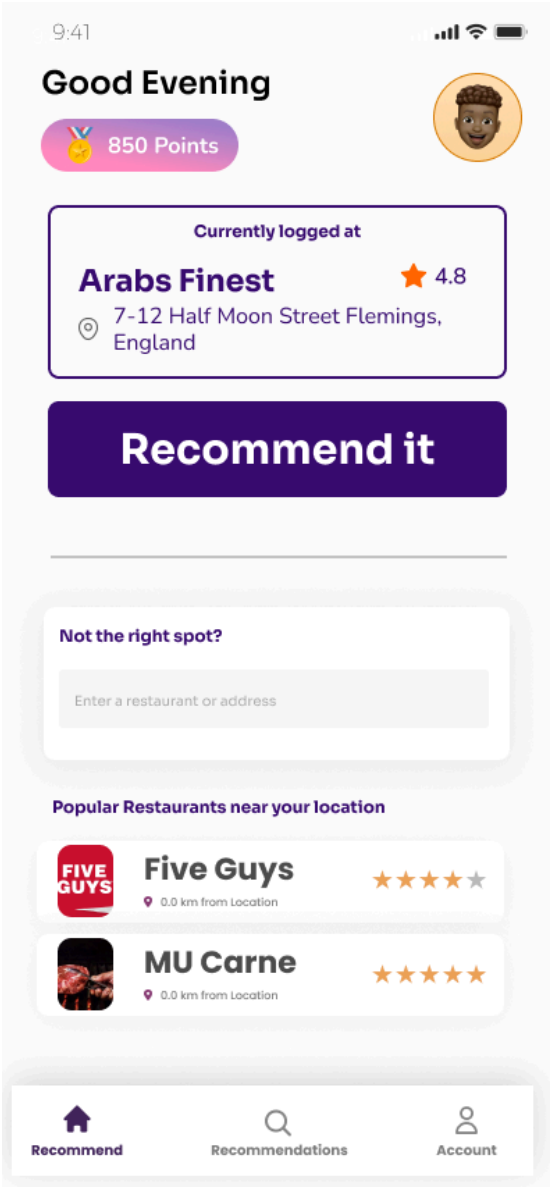
Find below the relevant explanations for all screens and functions in detail as well as explaining the journey and customer experience.

This section only talks about the front-end.

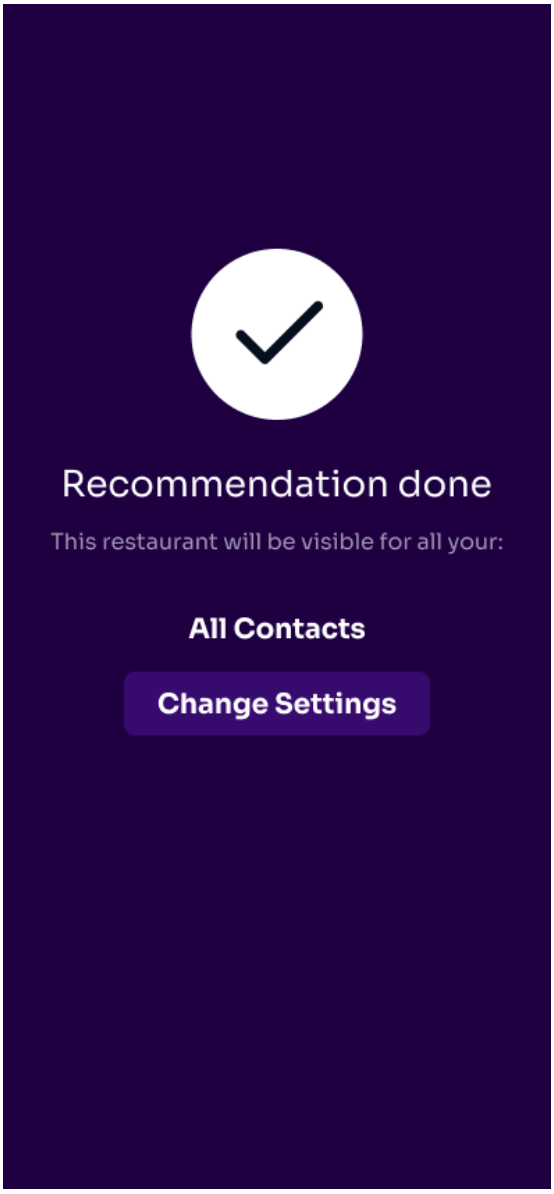
Login / Sign in

Overview	Explanation
	When not logged-in or signed in this is the screen the customer is seeing at first. It will require him to do a proper identification login, signin via his mobile phone number. The number may only be necessary for the first + We need a field displaying why we are asking for the number -> to have one identity and avoid fraud in the rating systems

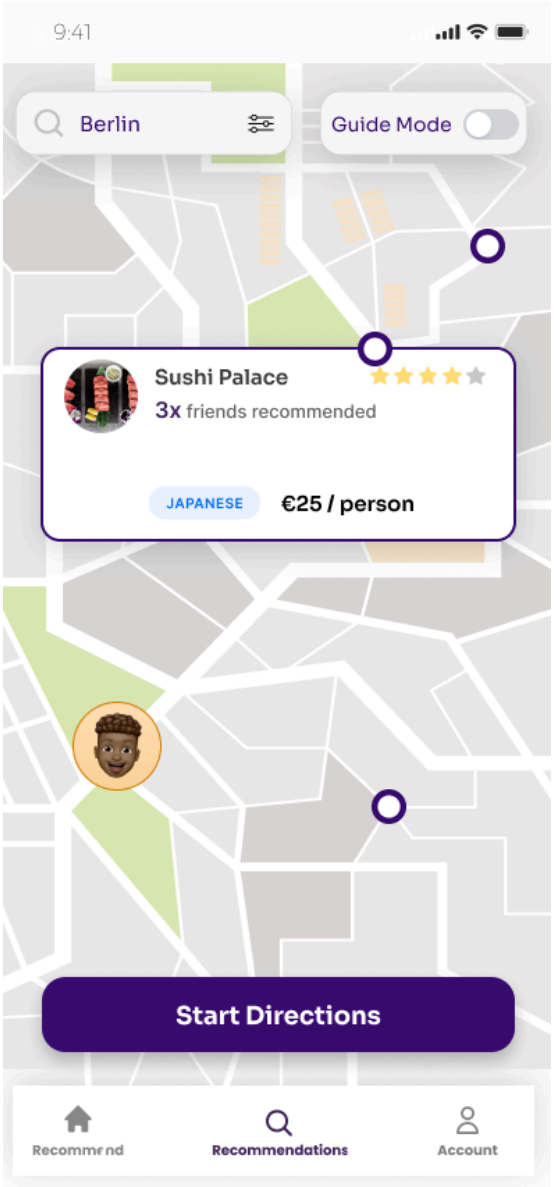
Home Screen

Overview	Explanation
	The most important screen. When a customer opens app, it needs to pre-populate the most probable restaurant, the user is at. From here it should be only one additional click to recommend the actual restaurant. If the location is not correct, the customer has the option to correct it via two ways. Either he decides to write the name of the restaurant or he uses pre-populated information (e.g. our 2, 3rd chances for most likely restaurant he is at or most popular restaurants near him) All the relevant information can be pulled from Google Maps API. The recommendation can happen anytime and from anywhere, so the user doesn't need to be technically near the restaurant. Behind the loyalty points there is no function.Please ignore in displaying, but definitely need it for the backend.

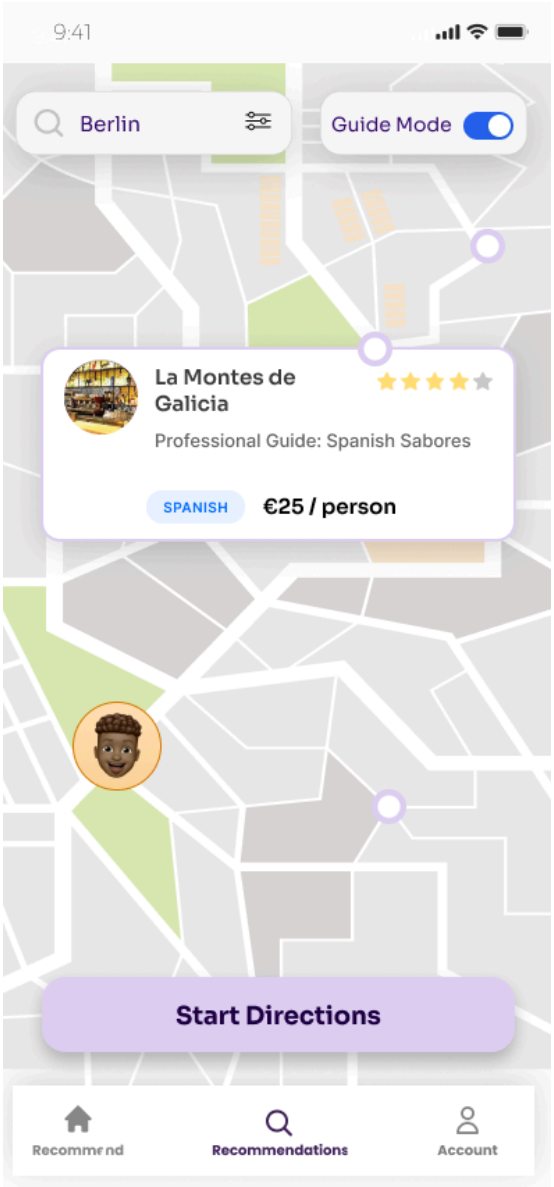
Making Recommendation Screen

Overview	Explanation
	After clicking the recommendation button from the home screen, the user receives a success message. The recommendation is at this point made. No need for further ratings. The user has the option to overwrite the setting of the pin / recommendation to certain lists. But for the MVP the standard will be always all contacts.

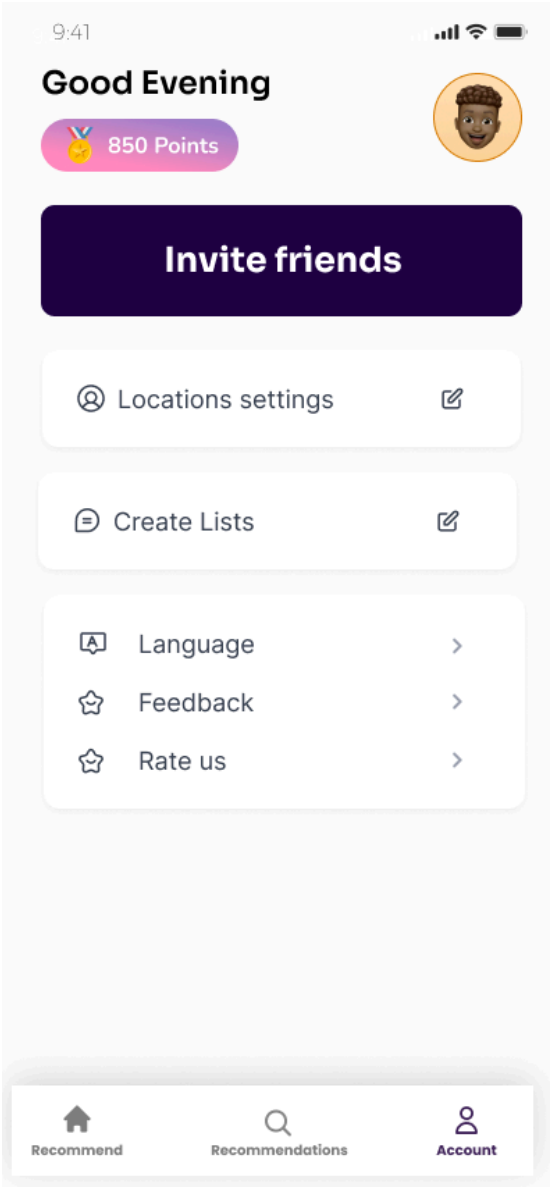
Search Mode -> Contacts

Overview	Explanation
	This should be your standard view for searching recommendations from your contacts. Ideally every pin contains information: - Who recommended it (name of contact) - Name of place - Opening hours - Google category of place - Google price range of place - Google rating of place - Main picture of place User has the ability to zoom out, zoom in and search for other cities to explore If he wants to start directions we hand over to Google Maps or Apple Maps after clicking the button, but we set a reminder in the backend to send a push notification +1 day if the user wants to recommend it and did not do so yet. User has also the ability to go into guide mode

Search Mode -> Guide Mode

Overview	Explanation
	In the beginning we are going to rely on some guides to emulate app usage and pre-populate some of the best list from guides around. Guide mode will only feature local guides, taste influencers or magazines like Michelin We work with the same information as in contacts mode, but will display instead of contact name: - the name of the guide - Link to the guide The rest stays the same as in contacts mode. Just the colors change to emulate a different environment.

Settings

Overview	Explanation
	Settings is a very plain overview of account settings. Here the user has the chance to: 1. Invite friends via sending link to contact list (not displayed in mock but well known) 2. Make location settings (standard phone settings?) 3. Create lists (Disabled for MVP) 4. Edit Language 5. Feedback 6. Rating 7. Deletion of Account (not displayed) Please ignore the loyalty points, this is just a placeholder for later functionality

Back-End Overview

The goal of the backend should be a flexible and dynamic system. We want to innovate on the data we are getting from the users and monetize later on.

Architectural Requierement #1 Microservices Approach

I would structure the backend into several core microservices, each responsible for a specific business capability. This promotes a separation of concerns and allows for independent development and scaling.

- **User Service:** This service would handle all user-related functionalities, including user registration, login, profile management, and authentication. It would manage user data like usernames, encrypted passwords, and profile pictures.
- **Restaurant Service:** This service would manage all restaurant-related data. This includes creating and updating restaurant profiles, menus, opening hours, and location information -> this service is probably super tied to the Google Maps API
- **Rating & Review Service:** This service would be dedicated to handling user ratings and reviews. It would manage the creation of pins and for whom they are visible or not.
- **Search & Discovery Service:** This service would power the app's search functionality. It would allow users to find restaurants based on various criteria like location and name in the beginning. This service is also tied to Google Maps API
- **Search & Discovery Service:** Take a look at Requierment #2

Architectural Requierement #2 Performance, Cost Oriented Approach

Initial Data Ingestion: When a new restaurant is added to the app's purview (e.g., a user searches for a place not in the database), this service would fetch its details from the Google Maps Places API.

Periodic Updates: It would periodically re-fetch data for existing restaurants to get the latest information, such as updated opening hours, new photos, or a change in its operational status (e.g., "Temporarily Closed"). This would run as a background job to avoid impacting user-facing performance.

Webhook Handler: To get real-time updates, I'd implement a webhook endpoint. While the Google Maps API doesn't directly offer webhooks for all changes, for certain aspects (or by using other Google Cloud services), you can be notified of updates, which would trigger this service to refresh the relevant data but probably nothing for the MVP and rather later stage.

Hybrid Data Model: The restaurant database schema would be designed to accommodate data from both Google Maps and the app's internal system. For instance, a restaurant entry would have fields for the Google Place ID, Google's rating, and also fields for the app's internal rating and user reviews.

Caching Layer

Since heavy reliance on an external API like Google Maps can get expensive and lead to rate limiting a robust caching layer is non-negotiable.

- **API Response Caching**: I would implement a caching layer (using a service like Redis) to store the responses from the Google Maps API. Before making a call to Google, the backend would first check if the requested data is already in the cache and is still valid. This would drastically reduce the number of API calls.
- **Tiered Caching Strategy**: A multi-level caching strategy could be employed. A hot cache for frequently accessed restaurants (e.g., popular places in a dense urban area) and a cold cache for less popular ones can be an option depending on the cost and complexity

---

Geocoding and Location-Based Service Enhancements

While the core Search & Discovery service would still exist, its internals would change.

- **Localized Search Proxy**: Instead of directly querying a local database for all location-based searches, this service would intelligently decide when to proxy a search to the Google Maps API (for discovering new places) and when to use its own database (for restaurants the app already knows about and has enriched with its own data).

Location Approximator

This algorithm can be a real cost driver when opening the app since it determines all the time, what is the most likely restaurant your currently at. The principle MUST be a cost effective approach. Thats why we provide an overview of how we may be able to safe on fees. For now the general process follows:

1. Real-time Location Acquisition 📍

The first step is getting the customer's location as accurately as possible. Three ways which I would run in an hierarchical order.

HTML5 Geolocation API (for web/mobile web) / Device Location SDK (for native apps): This is the primary method to get the user's GPS coordinates (latitude, longitude).

Google Maps Geolocation API (supplementary): For situations where GPS signal is weak or unavailable (e.g., deep indoors), this API can estimate location based on Wi-Fi access points and cell towers. This can provide a fallback, though usually with less precision than good GPS.

Passive Location Updates: Instead of aggressively polling GPS, leverage **passive location updates** from the device's operating system (e.g., Android's Fused Location Provider's `PRIORITY_LOW_POWER` or iOS's significant location change service). These use less battery and less frequent updates, which are cheaper.

And IMPORTANT Contextual Triggers: Only fence location when app is opened, 1x time.

2. Identifying Nearby Restaurants 🍽️

Once we have the customer's estimated location, we need to find potential restaurants.

Places API (New) - Nearby Search: This is the most suitable endpoint.

Parameters:

location: The customer's latitude, longitude.

radius: A reasonable radius (e.g., 50-200 meters) to encompass nearby restaurants. Too large, and you get too many irrelevant places; too small, and you miss potential candidates due to location inaccuracy.

includedType: Filter for restaurant, cafe, food, and other relevant eating establishment types to narrow down results.

rankPreference: DISTANCE is generally preferred here to get the closest options.

Field Masking: Request essential fields like placeId, displayName, location, address, and openingHours, priceLevel, user_ratings_total for further filtering.

Filtering:

Open Now: Filter out restaurants that are currently closed based on openingHours.

Relevance/Confidence Score: Consider the user_ratings_total or rating to give preference to more established or popular places.

3. Proximity and Confidence Scoring Algorithm 📊

This is where the "estimation" algorithm comes in. It needs to combine multiple signals to make an educated guess.

Distance to Centroid: Calculate the straight-line distance (Haversine formula) between the customer's reported location and the location of each nearby restaurant. Shorter distances generally indicate higher likelihood.

Accuracy Radius Overlap: Crucially, consider the accuracy radius provided by the geolocation source.

If the customer's accuracy circle largely contains or significantly overlaps with the restaurant's location, it increases confidence.

If the accuracy radius is very large and encompasses multiple restaurants, it reduces the confidence in any single restaurant.

4. Algorithm Steps & Refinements ⚙️

Step 1: Get Current Location

Step 2: Poll the user's device for latitude, longitude, and accuracy, or use Google Maps API

Step 3: Perform Nearby Search (Places API - Nearby Search): Use the current location as the location parameter.

Step 4: Set a radius (e.g., 20-50m).
Step 5: Filter includedTypes for restaurants.

Step 6: Request displayName, location (lat/lng), openingHours, priceLevel, rating using a field mask.

Step 7: Calculate Proximity Scores for Candidates:

For each returned restaurant:
Calculate the distance from the user's current latitude, longitude to the restaurant's location.
Factor in the user's accuracy radius. The closer the distance and smaller the accuracy radius, the higher the score.

Boost score if openingHours indicate it's open.

Factor in rating or user_ratings_total (a busy, highly-rated place might be a stronger candidate) -> e.g. 10-30%

Step 8: Identify Top Candidate(s):
Sort restaurants by their proximity score.

Consider a threshold: If the top candidate's score is significantly higher than others, and its distance is within a reasonable range (e.g., 10-20 meters) while factoring in accuracy, it's a strong estimate.

In this simplified model, **proximity is king** 👑. Rating and popularity are supporting actors that help when the primary signal is ambiguous, but they can't overcome a significant distance difference with just a single, static location point.

Function for Ingesting Backend Data

For the Guide mode, I would need a function which allows me to ingest data into the backend to set pins for well known and recommended pins which would be available for everybody

In a sense this would be just a mask where I would do the necessary input of data:

Name, Address of the Restaurant
Photo of Guide
Link to Guide

The rest of the data would be retrieved via the Google Maps API and then a recommendation would be set for all users in the Guide Mode

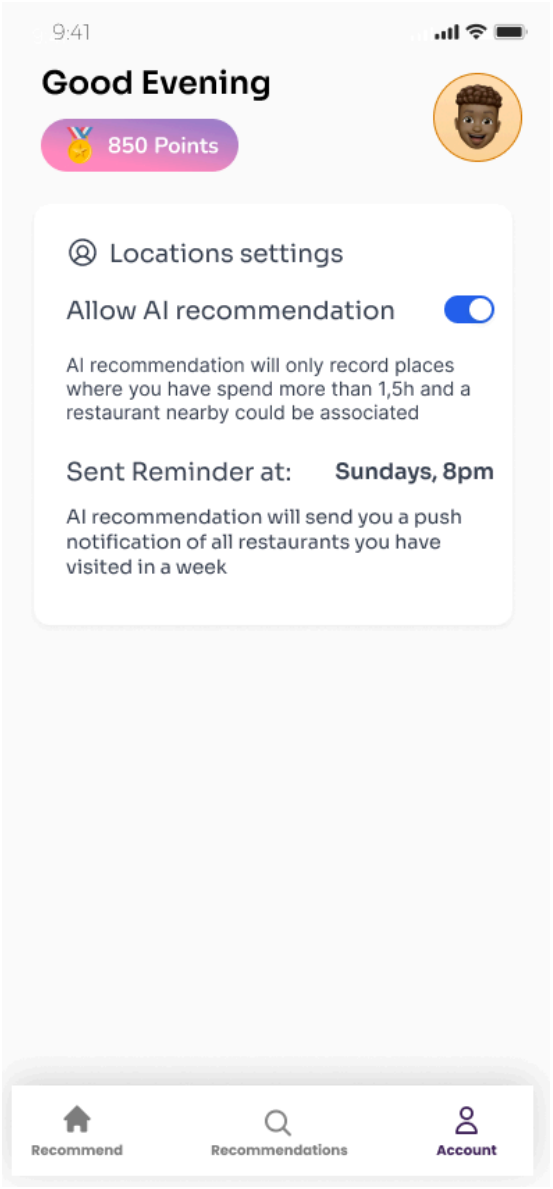
Looking beyond MVP

This is an aggregation of ideas to look beyond MVP

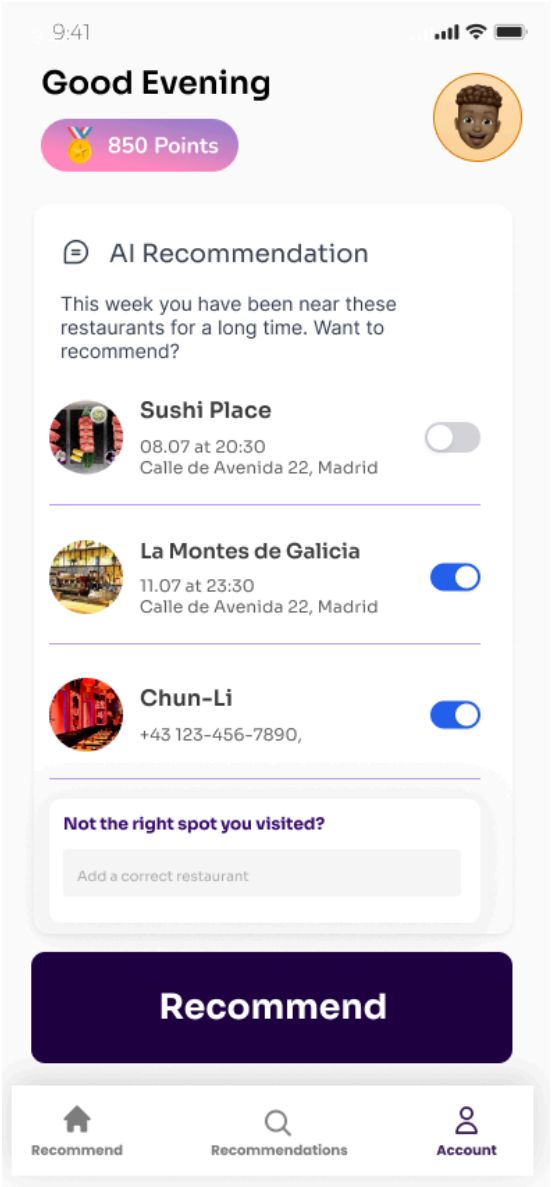
Not for MVP but for later discussion

Create List	List allows users to build pre-defined filters in order to steer recommendations. The standard is always, make available to all contacts, but user has function to specify distribution of recommendations according to pre-built lists based on his contacts
Add contact to list	E.g. like whatsapp groups but without chat function, just add them to list, they dont know they belong there
AI Recommendation	Special function which if allowed, will record places and use AI to determine whether you were in a restaurant or not. For start idea is to record only location where user spend more than 1,5 hours and a restaurant nearby could be associated -> algorithm simple required
In-App Rewards	Incentivize good recommendations

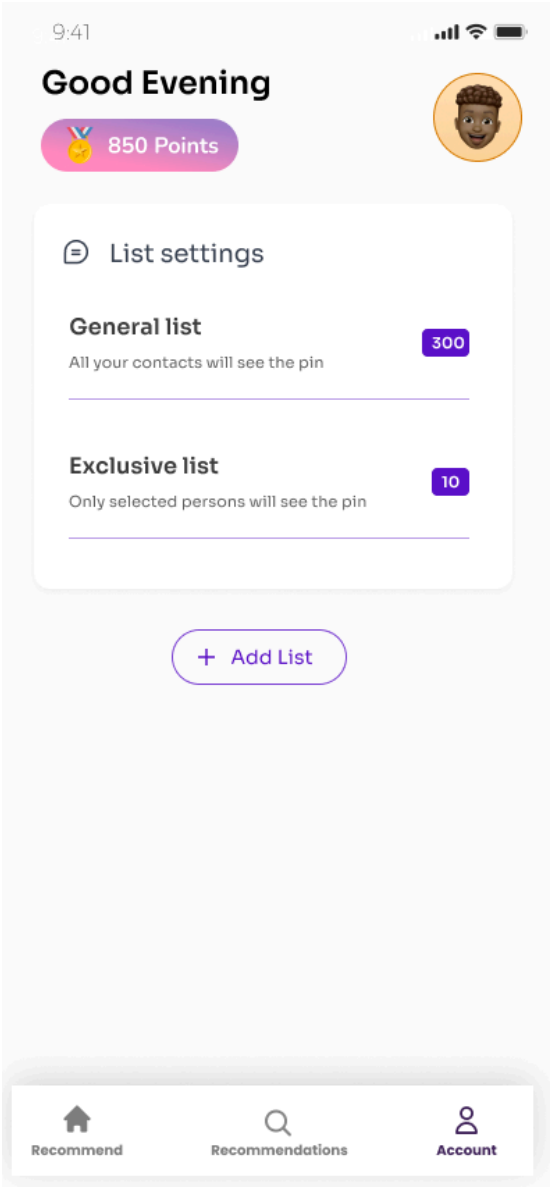
AI Recommendation Function

Overview	Explanation
	This app relies heavily on location tracking. So there are two functions. One is not displayed because its mostly handled by mobile phone side -> e.g. Apple settings or when Apple asks how often you want to location track. The other function is around a function called (working name) AI recommendation. AI recommendation will require constant tracking and its goal is to help and incentivize recommendations. If turned on, the algorithm will only detect restaurants where the customer spent more than 1,5 hours and log them for a subsequent push notification. We are fully aware that this function is not 100% exact but with the right taste model and enough data it can be. From the getgo we will highly rely on Googles popularity rating. User has also the ability to set push notifications when he wants to get the overview of all restaurants he has been visiting.

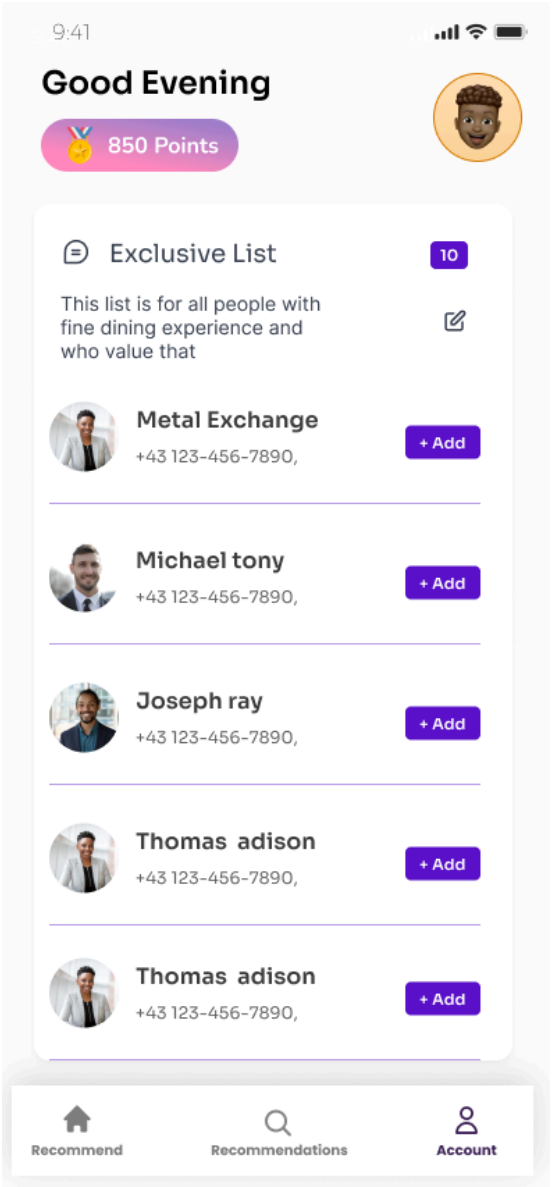
Location Tracking Push Notification

Overview	Explanation
	This would be the result of a week of tracking, assuming the user allowed permanent tracking. The algorithm only logs the most probable restaurants and provides the information. Plus there is also an ability to search for restaurants and add them to the list

Lists Overview

Overview	Explanation
	Lists is a way to filter to which contacts recommendations are pushed / recommendations are seeable. The general list is always available and there from the start (All his contacts) From there he can create specific lists.

Add List

Overview	Explanation
	When creating a list, user has the functionality to add contacts to the list (similar to whatsapp groups).